ACADEMIC TASK-2 INT332

(DEVOPS VIRTUALIZATION AND CONFIGURATION MANAGEMENT)

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Name: Abhijeet Kumar

Registration number: 12303503

Roll No. :- 01

Section: 40M59

Submitted to:

Divya Thakur



LOVELY PROFESSIONAL UNIVERSITY

A Project Report on

AI Resume Analyzer

An AI-Powered ATS Resume Analysis Tool

Built with Python · Flask · Groq AI · Docker · GitHub Actions

GitHub: github.com/abhijeetkumar51

Repository: github.com/abhijeetkumar51/ai-resume-analyzer

Table of Contents

Chapter 1 — Introduction

1.1 Project Overview

1.2 Problem Statement

1.3 Objectives

1.4 Scope

Chapter 2 — System Requirements

2.1 Hardware Requirements

2.2 Software Requirements

Chapter 3 — Workflow

Step-by-step process flow

Chapter 4 — Implementation

4.1 Project Setup Steps

4.2 Project Structure

4.3 Configuration Files

4.4 Application Code (app.py)

4.5 Dockerfile

4.6 GitHub Actions CI/CD (main.yml)

4.7 Docker Commands

4.8 GitHub Repository

Chapter 5 — Results and Output

5.1 Application Home Page

5.2 Resume Upload & Analysis

5.3 ATS Score Display

5.4 Detailed Analysis Results

5.5 CI/CD Build Success

5.6 Docker Image Creation

5.7 Running Container

5.8 Deployed Application

Chapter 6 — Future Scope

Chapter 7 — Conclusion

7.1 Objective Achieved

7.2 Technologies Learned

7.3 Final Outcome

Chapter 1 — Introduction

1.1 Project Overview

The AI Resume Analyzer is an intelligent, AI-powered web application that helps job seekers optimize their resumes for Applicant Tracking Systems (ATS). The application allows users to upload their resume in PDF format and paste a target job description. It then leverages the Groq AI API with the LLaMA 3.3 70B large language model to perform a comprehensive analysis.

The application generates the following outputs:

- ATS Compatibility Score (0–100)
- Matching Skills — skills the candidate already possesses
- Missing Skills — skills the candidate needs to add
- Strengths — strong points of the resume
- Weaknesses — areas that need improvement
- Improvement Suggestions — actionable recommendations
- Overall Summary — a concise 2–3 sentence assessment

The project is fully containerized with Docker, automated with GitHub Actions CI/CD, and deployed on Render for live access.

1.2 Problem Statement

In today's competitive job market, over 75% of resumes are rejected by Applicant Tracking Systems before they ever reach a human recruiter. Job seekers face several critical challenges:

1. Lack of ATS awareness — Most candidates are unaware that their resumes are first screened by automated software, not humans.
2. Keyword mismatch — Resumes often fail to include the specific keywords and skills mentioned in job descriptions.
3. No feedback loop — Candidates submit resumes without knowing why they are being rejected, receiving no actionable feedback.

4. Expensive professional review — Professional resume review services cost ₹2,000–₹10,000+ per session, making them inaccessible to many.
5. Generic resume tools — Existing free tools provide only basic formatting checks without AI-driven content analysis.

There is a clear need for a free, instant, and intelligent resume analysis tool that can compare a resume against a specific job description and provide detailed, actionable feedback.

1.3 Objectives

The primary objectives of this project are:

6. Build an AI-powered resume analyzer using a large language model (LLaMA 3.3 70B via Groq API) to evaluate resume-job description compatibility.
7. Extract text from PDF resumes automatically using the PyPDF2 library.
8. Generate a quantitative ATS score (0–100) that indicates how well a resume matches a job description.
9. Provide qualitative feedback including matching skills, missing skills, strengths, weaknesses, and suggestions.
10. Create a modern, responsive web interface using HTML, CSS, and JavaScript with a premium dark-mode design.
11. Containerize the application using Docker for consistent, reproducible deployments.
12. Automate the build and deployment pipeline using GitHub Actions CI/CD to push Docker images to Docker Hub.
13. Deploy the application on Render for public access.

1.4 Scope

In Scope:

- PDF resume upload and text extraction
- Job description text input
- AI-powered analysis using Groq API (LLaMA 3.3 70B model)

- ATS score generation with skill gap analysis
- Responsive, animated web UI with glassmorphism design
- Docker containerization
- CI/CD pipeline with GitHub Actions
- Cloud deployment on Render

Out of Scope:

- Support for non-PDF formats (DOCX, TXT, images)
- OCR for scanned/image-based PDFs
- User accounts, login, and history tracking
- Resume editing or rewriting
- Multi-language support (currently English only)
- Mobile native application

Chapter 2 — System Requirements

2.1 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB or more
Storage	2 GB free disk space	5 GB (including Docker images)
Network	Active internet connection	Broadband connection
Display	1024 × 768 resolution	1920 × 1080 or higher

Note: Since the heavy AI computation is performed server-side by the Groq cloud API, the local hardware requirements are minimal. Docker Desktop, if used, benefits from at least 8 GB RAM.

2.2 Software Requirements

Software	Version	Purpose
Python	3.11+	Backend runtime
pip	Latest	Python package manager
Flask	3.0.3	Web framework
Groq Python SDK	0.13.0	AI model API client
PyPDF2	3.0.1	PDF text extraction
Werkzeug	3.0.3	WSGI utility (Flask dependency)
python-dotenv	Latest	Environment variable loading
flask-cors	Latest	Cross-Origin Resource Sharing
Docker	20.10+	Containerization
Docker Desktop	Latest (optional)	Docker GUI for Windows/Mac
Git	2.30+	Version control
VS Code	Latest (optional)	Code editor / IDE
Web Browser	Chrome / Edge / Firefox	Frontend access
Groq API Key	Free tier	AI model access
GitHub Account	Free	Repository hosting and CI/CD
Docker Hub Account	Free	Docker image registry
Render Account	Free tier	Cloud deployment

Operating System: Windows 10/11, macOS, or Linux (Ubuntu 20.04+)

Chapter 3 — Workflow

The AI Resume Analyzer follows a clear step-by-step workflow from user interaction to AI-generated results. The complete process flow is described below:

Step 1 — User Interface Loading

The user navigates to http://localhost:5000. Flask serves the index.html template, which renders a premium dark-mode interface with animated background orbs, floating particles, and a glassmorphism card layout.

Step 2 — Resume Upload

The user drags and drops or clicks to upload a PDF resume file. The frontend validates that only .pdf files are accepted. The file name is displayed in a green confirmation badge.

Step 3 — Job Description Input

The user pastes the full job description text (including required skills, responsibilities, and qualifications) into the textarea.

Step 4 — Submit for Analysis

When the user clicks "Analyze My Resume," the frontend disables the button, shows an animated loader with step-by-step progress indicators, and sends a POST request with FormData to the /analyze endpoint.

Step 5 — Backend Validation

Flask validates that a PDF file is present, the filename is not empty, the file extension is .pdf, and the job description is not blank. If any validation fails, a 400 error is returned.

Step 6 — PDF Text Extraction

The uploaded PDF is saved to the uploads/ directory. PyPDF2 reads each page and extracts the text content. If the PDF is a scanned image with no extractable text, an error is returned.

Step 7 — AI Analysis via Groq

The extracted resume text and job description are embedded into a carefully crafted prompt. The prompt instructs the LLaMA 3.3 70B model to return a structured JSON object with exactly 7 fields: ats_score, matching_skills, missing_skills, strengths, weaknesses, suggestions, and summary.

Step 8 — Response Parsing

The raw AI response is cleaned (markdown code fences stripped if present) and parsed as JSON. The structured result is sent back to the frontend as a JSON response.

Step 9 — Results Rendering

The frontend renders the results with smooth animations: an animated ring chart for the ATS score, color-coded skill tags (green for matching, red for missing), and ordered lists for strengths, weaknesses, and suggestions.

Workflow Diagram:

User Opens App → Uploads Resume PDF → Pastes Job Description → Clicks Analyze → Frontend sends POST /analyze → Flask Validates Input → Saves PDF → PyPDF2 Extracts Text → Groq AI Analyzes → JSON Response → Frontend Renders Results

Chapter 4 — Implementation

4.1 Project Setup Steps

Step 1: Clone the Repository

```
git clone https://github.com/abhijeetkumar51/ai-resume-analyzer.git
cd ai-resume-analyzer
```

Step 2: Create a Virtual Environment

```
python -m venv venv
```

Step 3: Activate the Virtual Environment

```
# Windows (PowerShell)
.\venv\Scripts\Activate

# Mac/Linux
source venv/bin/activate
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

Step 5: Configure Environment Variables

Create a .env file in the project root:

```
GROQ_API_KEY=gsk_your_api_key_here
```

Get a free API key from: https://console.groq.com/keys

Step 6: Run the Application

```
python app.py
```

Open http://localhost:5000 in your browser.

4.2 Project Structure

```
ai-resume-analyser/
├── app.py                          # Flask backend + Groq AI integration
├── requirements.txt                # Python dependencies
├── Dockerfile                      # Docker container configuration
├── .dockerignore                   # Files excluded from Docker builds
├── .env                            # Environment variables (API key)
├── .env.example                    # Template for .env file
├── .gitignore                      # Files excluded from Git
├── README.md                       # Project documentation
├── templates/
│   └── index.html                  # Frontend UI (HTML + CSS + JS)
├── static/
│   └── .gitkeep                    # Keeps empty static folder in Git
├── uploads/                        # Temporary PDF storage (runtime)
└── .github/
    └── workflows/
        └── main.yml                # GitHub Actions CI/CD pipeline
```

4.3 Configuration Files

requirements.txt:

```
flask==3.0.3
groq==0.13.0
PyPDF2==3.0.1
Werkzeug==3.0.3
```

4.4 Application Code — app.py

The main application file (app.py) is the Flask backend that handles all server-side logic. Key components include:

- Environment loading — Reads the GROQ_API_KEY from the .env file using python-dotenv.
- Flask app initialization — Configures CORS, upload folder, and max file size (16 MB).
- get_api_key() — Reads the API key from environment variables at request time.
- extract_text_from_pdf() — Uses PyPDF2 to read each page of the uploaded PDF and extract text.
- analyze_resume_with_groq() — Constructs a detailed prompt, sends it to Groq's LLaMA 3.3 70B model, and parses the structured JSON response.
- Route "/" (GET) — Serves the frontend HTML page.
- Route "/analyze" (POST) — Handles resume upload, validation, text extraction, AI analysis, and returns JSON results.

Complete app.py source code:

```
import os
import json
import re
from dotenv import load_dotenv
from flask import Flask, request, render_template, jsonify
from flask_cors import CORS
from groq import Groq
import PyPDF2

load_dotenv()

app = Flask(__name__)
CORS(app, origins=['http://localhost:5173', 'http://127.0.0.1:5173'])
app.config['UPLOAD_FOLDER'] = 'uploads'
```

```
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024

os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

def get_api_key():
    return os.environ.get('GROQ_API_KEY', '').strip()

def extract_text_from_pdf(pdf_path):
    text = ""
    with open(pdf_path, 'rb') as file:
        pdf_reader = PyPDF2.PdfReader(file)
        for page in pdf_reader.pages:
            extracted = page.extract_text()
            if extracted:
                text += extracted + "\n"
    return text.strip()

def analyze_resume_with_groq(resume_text, job_description):
    api_key = get_api_key()
    if not api_key:
        raise Exception("GROQ_API_KEY is not set.")
    client = Groq(api_key=api_key)
    prompt = f"""You are an expert ATS and HR specialist.
Analyze the resume against the job description.
Return ONLY a valid JSON object with: ats_score, matching_skills,
missing_skills, strengths, weaknesses, suggestions, summary.

RESUME: {resume_text}
JOB DESCRIPTION: {job_description}"""

    chat_completion = client.chat.completions.create(
        messages=[
            {"role": "system", "content": "You are an expert ATS resume
analyzer."},
            {"role": "user", "content": prompt}
        ],
        model="llama-3.3-70b-versatile",
        temperature=0.3,
        max_tokens=2048,
```

```
    )
    raw = chat_completion.choices[0].message.content.strip()
    raw = re.sub(r'^```(?:json)?\s*', '', raw)
    raw = re.sub(r'\s*```$', '', raw)
    return json.loads(raw)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/analyze', methods=['POST'])
def analyze():
    if 'resume' not in request.files:
        return jsonify({'error': 'No resume file uploaded.'}), 400
    file = request.files['resume']
    job_description = request.form.get('job_description', '').strip()
    if not file.filename.lower().endswith('.pdf'):
        return jsonify({'error': 'Only PDF files are supported.'}), 400
    pdf_path = os.path.join(app.config['UPLOAD_FOLDER'], 'resume.pdf')
    file.save(pdf_path)
    resume_text = extract_text_from_pdf(pdf_path)
    if not resume_text:
        return jsonify({'error': 'Could not extract text.'}), 400
    try:
        result = analyze_resume_with_groq(resume_text, job_description)
        return jsonify(result)
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

4.5 Dockerfile

```
# Use latest patched Python slim image
FROM python:3.13-slim
```

```
# Patch OS-level vulnerabilities in base image
RUN apt-get update && apt-get upgrade -y && apt-get clean && rm -rf
/var/lib/apt/lists/*

# Set working directory inside container
WORKDIR /app

# Copy requirements first (layer caching - faster rebuilds)
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy all project files
COPY . .

# Create uploads directory inside container
RUN mkdir -p uploads

# Expose port 5000 to the host
EXPOSE 5000

# Run the Flask app
CMD ["python", "app.py"]
```

Dockerfile Instruction Explanation:

Instruction	Purpose
FROM python:3.13-slim	Uses a minimal Python 3.13 base image
RUN apt-get update && upgrade	Patches OS-level security vulnerabilities
WORKDIR /app	Sets /app as the working directory inside the container
COPY requirements.txt .	Copies dependencies file first for Docker layer caching
RUN pip install ...	Installs Python packages without caching to

	save space
COPY . .	Copies all project files into the container
RUN mkdir -p uploads	Creates the uploads directory for temporary PDF storage
EXPOSE 5000	Documents that the container listens on port 5000
CMD ["python", "app.py"]	Starts the Flask application when the container runs

4.6 GitHub Actions CI/CD — main.yml

```
name: CI/CD - Build & Push Docker Image

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
```

```
    - name: Log in to Docker Hub
      if: github.event_name == 'push'
      uses: docker/login-action@v3
      with:
        username: abhijeetchy51
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build Docker image
      run: docker build -t abhijeetchy51/ai-resume-analyser:latest .

    - name: Push to Docker Hub
      if: github.event_name == 'push'
      run: docker push abhijeetchy51/ai-resume-analyser:latest
```

CI/CD Pipeline Steps:

Step	Description
Trigger	Runs on every push or pull_request to the main branch
Checkout	Clones the repository code using actions/checkout@v4
Python Setup	Installs Python 3.11 for dependency validation
Install Dependencies	Upgrades pip and installs all packages from requirements.txt
Docker Login	Authenticates with Docker Hub using stored secrets (only on push)
Build Image	Builds the Docker image tagged as abhijeetchy51/ai-resume-analyser:latest
Push Image	Pushes the built image to Docker Hub for deployment (only on push)

4.7 Docker Commands

```
# Build the Docker image locally
docker build -t abhijeetchy51/ai-resume-analyser:latest .

# Run the container with API key
docker run -p 5000:5000 -e GROQ_API_KEY="gsk_your_key_here" abhijeetchy51/ai-resume-analyser:latest

# Pull the pre-built image from Docker Hub
docker pull abhijeetchy51/ai-resume-analyser:latest

# List running containers
docker ps

# Stop a running container
docker stop <container_id>
```

4.8 GitHub Repository

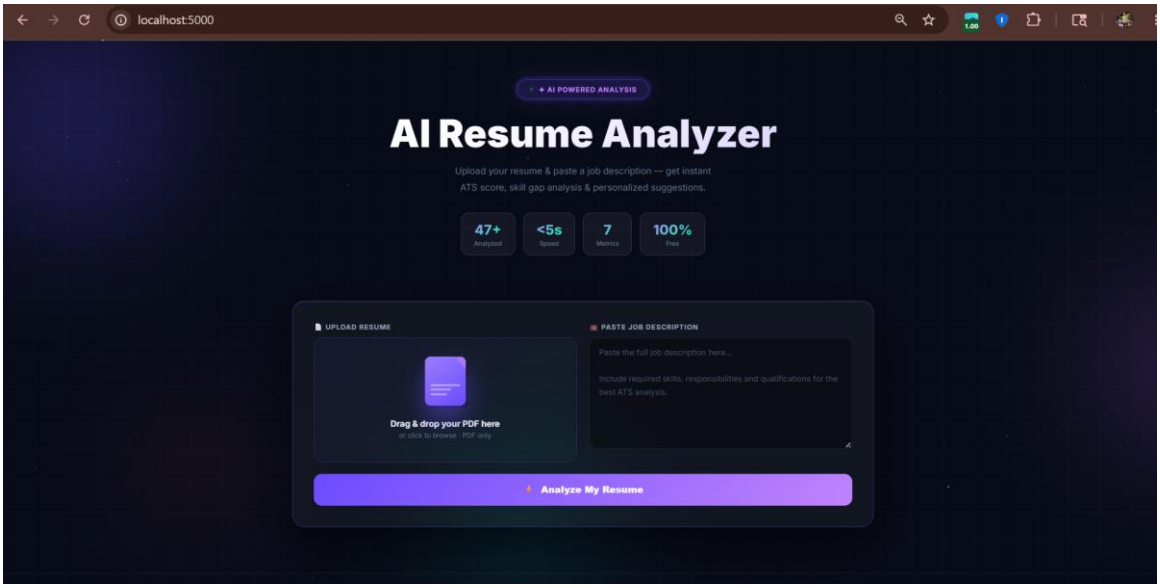
- Repository URL: https://github.com/abhijeetkumar51/ai-resume-analyzer
- Docker Hub: https://hub.docker.com/r/abhijeetchy51/ai-resume-analyser
- License: MIT

Chapter 5 — Results and Output

5.1 Application Home Page

When the user opens http://localhost:5000, they are greeted with a premium dark-mode interface featuring:

- Animated gradient background with floating orbs and particles
- A glassmorphism card with a 3D animated document icon for PDF upload
- A job description textarea
- An "Analyze My Resume" button with a shimmer hover effect
- Statistics bar showing analysis count, speed (<5s), metrics (7), and price (100% Free)



[Application Home Page]

5.2 Resume Upload & Analysis

After the user uploads a PDF and pastes a job description:

14. The submit button changes to "Analyzing…" and becomes disabled.
15. A multi-step loader appears with animated progress steps: Extracting resume content → Matching with job requirements → Calculating ATS score → Building improvement plan.
16. The analysis completes in under 5 seconds.

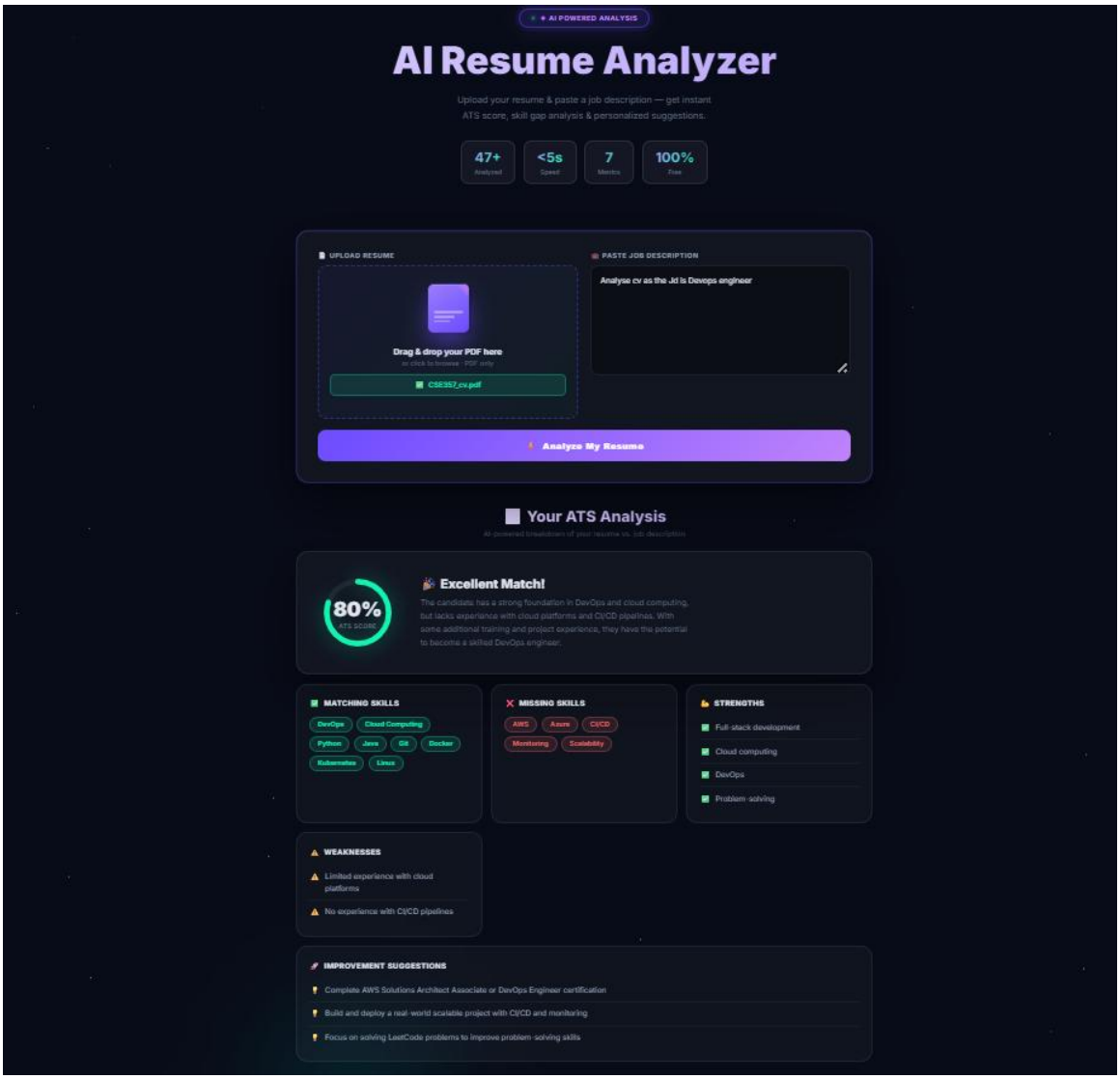
5.3 ATS Score Display

The result section shows a large animated ring chart with the ATS score:

- 70–100 (Green): "Excellent Match!" — strong alignment with the job
- 45–69 (Yellow): "Room to Improve" — partial match with gaps
- 0–44 (Red): "Needs Work" — significant skill gaps

The score number animates from 0 to the final value with a smooth counter effect.

[ATS Scoring Chart]



5.4 Detailed Analysis Results

The results section displays six cards in a responsive grid:

Card	Content	Visual Style
ATS Score	Ring chart + score + summary text	Animated SVG ring with glow
Matching Skills	Skills the candidate has	Green tags with hover effect
Missing Skills	Skills to add to resume	Red tags with hover effect
Strengths	Resume strong points	Green bullet list

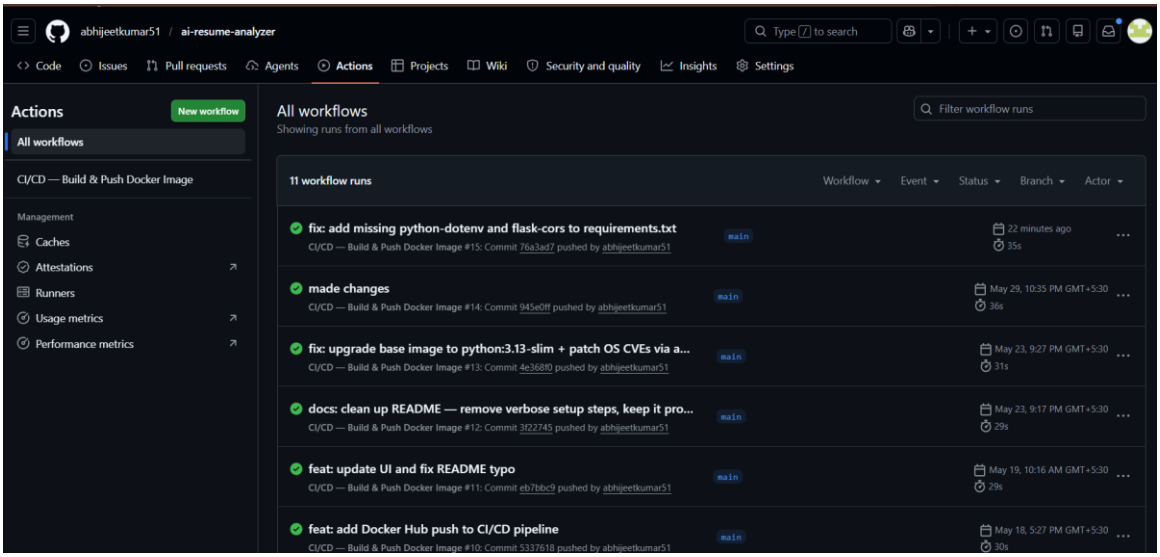
Weaknesses	Areas needing improvement	Warning bullet list
Suggestions	Actionable improvement tips	Lightbulb bullet list

5.5 Successful GitHub Actions CI/CD Build

When code is pushed to the main branch, the GitHub Actions pipeline executes:

17. Checks out the code — ☑ Success
18. Sets up Python 3.11 — ☑ Success
19. Installs all dependencies — ☑ Success
20. Logs into Docker Hub — ☑ Success
21. Builds the Docker image — ☑ Success
22. Pushes the image to Docker Hub — ☑ Success

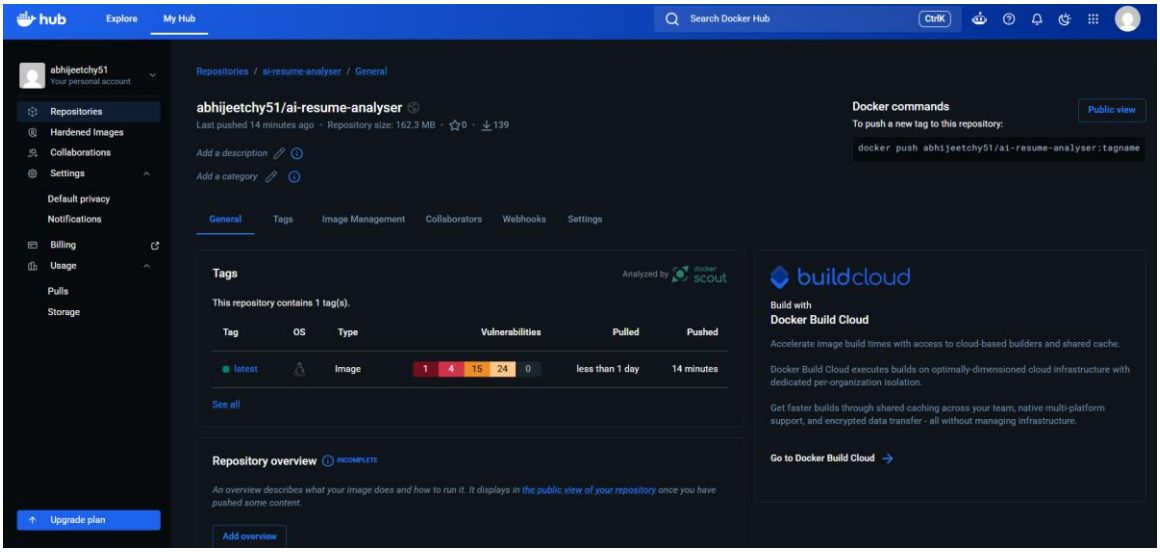
[GitHub Actions Successful Build (Green Checkmark)]



5.6 Docker Image Creation

Property	Value
Image Name	abhijeetchy51/ai-resume-analyser:latest
Base Image	python:3.13-slim
Exposed Port	5000
Registry	Docker Hub

[Docker Hub showing the pushed image]



5.7 Running Container

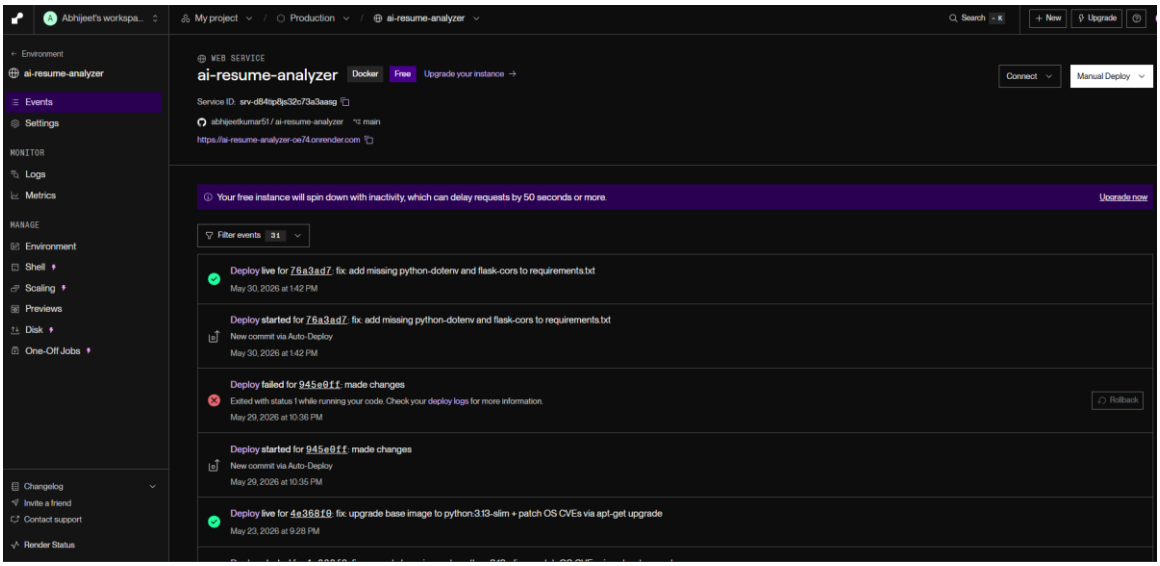
When the Docker container runs, the following output is displayed:

```
* Serving Flask app 'app'
* Debug mode: on
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.x.x.x:5000
```

5.8 Deployed Application

The application is deployed on Render and is publicly accessible. The deployment process:

23. GitHub Actions builds and pushes the Docker image to Docker Hub.
24. Render pulls the latest Docker image from Docker Hub.
25. The container starts with the GROQ_API_KEY injected as a runtime environment variable.
26. The application is live and accessible via the Render URL.



Chapter 6 — Future Scope

The AI Resume Analyzer has significant potential for future enhancements:

6.1 Multi-Format Support

- Support for DOCX, TXT, and RTF file formats using python-docx and similar libraries.
- OCR integration (Tesseract) for scanned/image-based PDFs.

6.2 User Accounts & History

- User registration and login system.
- Resume analysis history dashboard.
- Progress tracking across multiple job applications.

6.3 AI-Powered Resume Rewriting

- Automatically suggest rewritten bullet points optimized for ATS.
- Generate tailored cover letters based on the resume and job description.
- Keyword injection tool that naturally weaves missing keywords into the resume.

6.4 Multi-Language Support

- Support for resume analysis in Hindi, Spanish, French, German, and other languages.
- Multilingual job description matching.

6.5 Advanced Analytics

- Industry benchmarking — compare scores against industry averages.
- Trend analysis — track how resume quality improves over time.
- Skill demand heatmaps — visualize in-demand skills by industry and region.

6.6 Integration & APIs

- LinkedIn profile import for direct analysis.
- Chrome browser extension for one-click analysis from job portals.
- Public REST API for third-party integrations.

6.7 Mobile Application

- Native Android and iOS apps using React Native or Flutter.
- Camera-based resume scanning with real-time analysis.

6.8 Enhanced Deployment

- Kubernetes orchestration for horizontal scaling.
- Redis caching to reduce redundant API calls.
- Rate limiting and API key management for public deployment.

Chapter 7 — Conclusion

7.1 Objective Achieved

The AI Resume Analyzer project has successfully achieved all of its primary objectives:

- ☑ Built an AI-powered resume analyzer — The application uses the Groq API with the LLaMA 3.3 70B model to perform intelligent, context-aware resume analysis.
- ☑ PDF text extraction — PyPDF2 reliably extracts text from uploaded PDF resumes.
- ☑ ATS score generation — The system generates a quantitative compatibility score (0–100) along with seven detailed metrics.
- ☑ Modern web interface — A premium, responsive UI with dark mode, glassmorphism, animations, and a polished user experience.
- ☑ Docker containerization — The application is fully containerized with a production-ready Dockerfile.
- ☑ CI/CD automation — GitHub Actions automatically builds and pushes Docker images on every push to main.
- ☑ Cloud deployment — The application is deployed and publicly accessible on Render.

7.2 Technologies Learned

Through this project, the following technologies and concepts were learned and applied:

Technology	Skill Acquired
Python & Flask	Building REST APIs, handling file uploads, serving templates
Groq API & LLMs	Prompt engineering, structured JSON output, AI integration
PyPDF2	Programmatic PDF text extraction
HTML/CSS/JS	Modern UI design, CSS animations,

	glassmorphism, responsive layouts
Docker	Writing Dockerfiles, building images, running containers, layer caching
GitHub Actions	CI/CD pipelines, automated builds, Docker Hub integration
Render	Cloud deployment of containerized applications
Environment Variables	Secure API key management with .env files
Git & GitHub	Version control, branching, .gitignore, repository management

7.3 Final Outcome

The AI Resume Analyzer is a fully functional, production-ready web application that demonstrates the practical integration of modern AI capabilities with web development, containerization, and DevOps practices. It solves a real-world problem by providing job seekers with instant, free, and actionable feedback on their resumes — helping them improve their chances of clearing ATS filters and landing interviews.

The project showcases end-to-end software development skills: from designing a premium user interface to building a robust backend, containerizing with Docker, automating deployments with CI/CD, and deploying to the cloud.

— End of Report —